

Assignment No: - 8

Name :- Isha Ghorpade

Enrollment :- 22420020

Roll :- 381066

Problem Statement

Fine-tune a pre-trained transformer model for one NLP task such as Text Classification, Neural Machine Translation, or Text Summarization using a relevant dataset. Communicate the project outcomes through a detailed report and professional presentation.

Objectives

- To understand the architecture and working of transformer-based models.
 - To fine-tune a pre-trained transformer model for an NLP task.
 - To train the model using a real-world dataset.
 - To evaluate the model using performance metrics.
 - To present the project results through a report and presentation.
-

Software and Hardware Requirements

Software

- Python 3.x
- Jupyter Notebook / Google Colab
- Libraries:
 - Transformers (HuggingFace)
 - PyTorch / TensorFlow
 - Pandas
 - Scikit-learn
 - NumPy

Hardware

- Computer or Laptop
 - Minimum 8 GB RAM recommended
 - GPU (optional but faster training)
-

Theory

Transformer Models

Transformers are deep learning models designed for Natural Language Processing tasks. They use a mechanism called self-attention to process words in parallel instead of sequentially.

Transformers were introduced in the paper:

“Attention Is All You Need” (2017)

Popular transformer models include:

Model	Developed By
BERT	Google
GPT	OpenAI
RoBERTa	Facebook
T5	Google

Pre-Trained Models

A pre-trained model is trained on a large dataset beforehand.

Instead of training from scratch, we fine-tune the model on a smaller task-specific dataset.

Advantages:

- Saves training time
- Requires less data
- Achieves better performance

Fine-Tuning

Fine-tuning means slightly adjusting the weights of a pre-trained model for a specific task.

Example tasks:

- Sentiment classification
- Language translation
- Text summarization
- Question answering

Selected Task: Text Classification

Text classification assigns categories to text documents.

Example:

Input:

This movie is amazing!

Output:

Positive Sentiment

Dataset Used

Example dataset:

IMDb Movie Reviews Dataset

Contains:

Field	Description
-------	-------------

Review	Movie review text
--------	-------------------

Label	Positive / Negative sentiment
-------	-------------------------------

Dataset size:

- 50,000 reviews
-

Methodology

Steps followed in the project:

1. Load dataset
 2. Perform preprocessing
 3. Tokenize text using transformer tokenizer
 4. Load pre-trained model
 5. Fine-tune model on dataset
 6. Evaluate performance
 7. Analyze results
-

Workflow

Dataset



Preprocessing



Tokenizer



Pre-trained Transformer (BERT)

↓
Fine-tuning
↓
Prediction
↓
Evaluation

Implementation (Python Code)

Example using BERT for Text Classification

```
from transformers import BertTokenizer, BertForSequenceClassification
from transformers import Trainer, TrainingArguments
from datasets import load_dataset

# Load dataset
dataset = load_dataset("imdb")

# Load tokenizer
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

# Tokenization function
def tokenize(example):
    return tokenizer(example['text'], padding="max_length", truncation=True)

tokenized_datasets = dataset.map(tokenize, batched=True)

# Load pre-trained model
model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)

# Training arguments
training_args = TrainingArguments(
    output_dir="./results",
    evaluation_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    num_train_epochs=2
)

# Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["test"]
)
```

)

```
# Train model
trainer.train()
```

Evaluation Metrics

Common evaluation metrics include:

Metric	Description
Accuracy	Correct predictions / total predictions
Precision	Correct positive predictions
Recall	Ability to find all relevant samples
F1 Score	Balance between precision and recall

Expected Results

Example output:

Metric	Value
Accuracy	92%
Precision	91%
Recall	90%
F1 Score	90.5%

Applications

Transformer fine-tuning is used in:

- Chatbots
 - Sentiment analysis
 - Machine translation
 - Document summarization
 - Question answering systems
-

Conclusion

In this project, a pre-trained transformer model (BERT) was fine-tuned for a text classification task using a real-world dataset. The model demonstrated strong performance due to the powerful contextual understanding of transformer architectures. Fine-tuning pre-trained models significantly reduces training time while improving accuracy in NLP tasks.
